

FHIR R4 Expert Due Diligence

What a FHIR expert will ask before trusting PilotFish
as the FHIR server for their organization

PilotFish FHIR R4 Expandable Platform · CapabilityStatement 0.6.0 · 2026-08-05

How to read this document. This is not a sales brochure. It explains what a real FHIR server is expected to do, maps those expectations onto this demo, and answers the hard questions a FHIR architect, ONC/USHIK reviewer, or EHR integration lead would ask. Gaps are named explicitly so buyers can decide if PilotFish is the right *primary* FHIR store, a *facade/orchestrator*, or both over time.

1. What “a FHIR server” actually means

People say “FHIR server” casually. Experts mean a system that implements a large slice of the **HL7 FHIR HTTP API** (R4 = version 4.0.1), exposes an accurate **CapabilityStatement** at /metadata, stores (or proxies) resources with correct HTTP semantics, and participates in an ecosystem of clients, IGs (Implementation Guides), security profiles, and conformance tests. It is not “JSON over REST with Patient in the path.”

Core expectations experts bring into any evaluation:

- **Conformance:** CapabilityStatement matches real behavior; undeclared features must not silently half-work.
- **Resource lifecycle:** create / read / update / delete / history / vread; version ids and ETags; conditional create/update/delete when claimed.
- **Search:** typed parameters, modifiers, prefixes, chaining, includes, revIncludes, paging (_count, next links), and result Bundle semantics.
- **Transactions:** Bundle type=transaction and batch with atomicity rules and correct response entries.
- **Validation:** structural + terminology + profile/IG validation (base R4 is the floor, not the ceiling).
- **Security:** TLS, OAuth2/OIDC, SMART on FHIR scopes, audit (Provenance / AuditEvent), least privilege.
- **Bulk Data:** HL7 Bulk Data Access IG (async \$export, NDJSON, status, kickoff semantics).
- **Operations:** named ops (\$everything, \$validate, \$export, ...) with correct Parameters/Binary/OperationOutcome patterns.
- **Operational maturity:** multi-node, backups, DR, tenancy, rate limits, observability, upgrade path.

Reference-class open-source servers (e.g. HAPI FHIR JPA, Firely/Spark, Microsoft FHIR Server) optimize for being the **system of record for FHIR resources**. Middleware like PilotFish typically shines as a **facade, orchestrator, and integration fabric** — transforming, routing, securing, and composing systems — which may or may not also persist FHIR documents. That distinction is the root of most expert skepticism.

2. What this PilotFish demo is (and is not)

Dimension	This demo (Phase 1–6)	Typical “full” FHIR server
Role	EIP facade + SQL persistence for selected resources	Dedicated FHIR persistence engine + search index
CRUD	Patient, Observation, Condition, Encounter (+ metadata)	Broad resource catalog (often 50–100+ types)
Search	Targeted token/string params (core-6 style), not full grammar	FHIR search chap. + modifiers + chaining + includes
History / vread	Deferred	Usually required for clinical systems of record
Validation	HAPI validator, base R4 profiles	Base + IG packages (US Core, Da Vinci, ...)
Auth	Keycloak Bearer on write + Bulk; open reads in demo	SMART App Launch + scopes + sometimes backend services
Bulk \$export	Async system-level export → NDJSON (demo partitions)	Patient/Group/system export, _since, deletion markers
IG / US Core	Not packaged	Required for many US payer/provider programs
CapStatement	Draft 0.6.0 — intentionally scoped	Production-grade, tested against Inferno/Touchstone

One-sentence truth: This platform proves PilotFish can *speak FHIR R4* end-to-end (HTTP semantics, validation, OAuth Bearer, Bundle, Bulk export) with honest CapabilityStatement scoping — it does **not** claim parity with HAPI/Firely as a general-purpose FHIR repository.

3. Questions a true FHIR expert will ask

Below are the questions that separate “we put Patient JSON on a URL” from a viable FHIR program. Answers are candid about **this demo** and what a production PilotFish program would still need.

3.1 Conformance & truth in advertising

Q: Does your CapabilityStatement accurately describe what the server does — and refuse what it doesn't?

A: Experts treat CapStatement as a contract. This demo publishes a draft CapStatement (0.6.0) that lists implemented interactions and explicitly defers full search grammar, history, US Core packages, Group/Patient compartment export, CapStatement \$validate, and full SMART launch. That honesty matters. A deal-breaker is CapStatement inflation. Production stance: keep CapStatement generated from runtime truth (or rigorously reviewed against Inferno/Touchstone), never marketing copy.

Q: Have you run official conformance suites (Inferno, Touchstone, Crucible) against this endpoint?

A: Not claimed for this demo. Experts will ask for suite evidence before accepting PilotFish as a certified FHIR surface for CMS/ONC-facing programs. Path: scope CapStatement → lock behavior → run Inferno modules that match declared features → fix deltas before claiming production readiness.

3.2 Persistence model & identity

Q: Is PilotFish the system of record, or a facade over another clinical store?

A: This demo persists selected resources in SQL via PilotFish listeners — it is a self-contained facade+store for demo resources. Experts will ask which production model you intend: (A) PilotFish as SoR for FHIR documents, (B) PilotFish façade over EHR/CDR with on-the-fly mapping, or (C) hybrid (cache + origin). Each has different identity, conflict, and deletion rules. Clarify this before search and history conversations.

Q: How are resource ids, versions (meta.versionId), and ETags managed on concurrent updates?

A: A production FHIR server needs strict versioning and often optimistic concurrency (If-Match / versionId). This demo implements CRUD and soft delete but defers full history/vread semantics. Experts will press until versioning is explicit — especially if clinical apps rely on lost-update protection.

Q: What happens on DELETE — hard delete, soft delete, or FHIR deletion markers for Bulk _since?

A: Demo uses soft-delete style retention for CRUD. Bulk Data IG consumers often require deletion markers when using _since. That behavior is explicitly deferred. Call it out in any payer/Bulk RFP.

3.3 Search — the #1 expert trap

Q: Do you implement the FHIR Search specification, or a curated subset?

A: Curated subset — by design in Phase 1–6. Full FHIR search includes parameter types, modifiers (:exact, :contains, :missing, ...), prefixes (ge/le), composites, chaining (subject.name), _include/_revinclude, _sort, _summary, and paging. Experts reject “we have ?name=” as search. Viable path with PilotFish: declare only supported params in CapStatement; implement a search index (SQL/Elastic/OpenSearch) for declared params; expand deliberately per IG (e.g. US Core required searches) rather than pretending full grammar exists.

Q: How do you guarantee CapStatement searchParam lists stay in sync with the implementation?

A: This is a governance problem. Experts have been burned by CapStatements that advertise parameters the server ignores. Recommend: generate CapStatement fragments from the same config that drives the search listener, plus automated tests that probe each declared parameter.

3.4 Transactions, integrity & consistency

Q: Are Bundle type=transaction requests truly atomic?

A: FHIR transaction Bundles require all-or-nothing semantics (with precise failure reporting). Experts will ask whether PilotFish wraps SQL transactions correctly across multi-resource writes, how referential integrity is handled for literal references vs urn:uuid placeholders, and how partial batch failures are reported. Demo implements transaction/batch handling — production buyers should request failure-mode tests (constraint violation mid-Bundle).

Q: Do you support conditional create/update/delete and If-None-Exist?

A: Often required by EHR sync patterns. If not implemented, CapStatement must not advertise them. Experts will probe with Prefer headers and conditional URL syntax early in an RFP.

3.5 Validation, profiles & IGs

Q: Is validation base FHIR R4 only, or do you load IG packages (US Core, Da Vinci, CARIN)?

A: Demo: HAPI validator with base R4. That catches structural and many cardinality bugs. It does **not** prove US Core Patient/US Core Observation conformance. US programs almost always require IG package loading, StructureDefinitions, ValueSets, and often terminology services. PilotFish can integrate HAPI's IG support or call an external validator — but packaging IGs is a deliberate project, not a checkbox.

Q: Do you expose \$validate and refuse invalid writes under a profile?

A: CapStatement \$validate is deferred. Experts distinguish “we can validate in a pipeline” from “clients can POST to \$validate” and “server rejects non-conformant creates.” Decide policy: warn vs hard-fail; document it.

3.6 Security & SMART

Q: Is this SMART App Launch compliant, or just “OAuth Bearer somewhere”?

A: Demo uses Keycloak Bearer for writes and Bulk export; reads are intentionally open for demo friction reduction. SMART adds launch context, scopes (patient/*.read, system/*.rs), well-known endpoints, PKCE, and often EHR launch vs standalone. Experts will reject “we put Keycloak in front” as SMART. Production: either implement SMART correctly or position PilotFish behind an API gateway that already does SMART and pass identity claims into EIP.

Q: How are scopes enforced per resource and per compartment?

A: Token presence ≠ authorization. patient/Observation.rs must not return another patient's data. Experts will ask for compartment isolation tests. Demo is not positioned as a multi-tenant SMART authorization engine — that must be designed (claims → patient compartment filters).

Q: Where is the audit trail (AuditEvent / Provenance) for clinical forensic requirements?

A: Not a Phase 1–6 claim. Hospitals and payers often require who-read-what and who-changed-what. PilotFish can emit AuditEvent resources or SIEM logs — experts will ask which, with retention.

3.7 Bulk Data Access

Q: Is \$export aligned with the HL7 Bulk Data Access IG (kickoff, status, complete, NDJSON)?

A: Demo implements async system-level \$export with 202 + Content-Location, status polling, and NDJSON file download behind Bearer auth — the right shape. Experts will still ask about: Patient- and Group-level export, _typeFilters, _since with deletion markers, multi-node job coordination, TLS file URLs, and backend-services authorization (client_credentials + scopes). Those are the usual production deltas from a working demo.

3.8 Performance, scale & operations

Q: Can this survive EHR-scale read volume and nightly Bulk extracts?

A: Search and Bulk are index/IO problems. A generalist integration engine + row store can struggle if you treat FHIR as “dump JSON columns and SELECT LIKE.” Experts want: search indexing strategy, pagination cost, export job isolation from interactive API, horizontal scale of EIP vs DB, and SLOs. PilotFish viability improves when FHIR interactive API is bounded and heavy extract work is async (as this demo starts to show with \$export).

Q: How do you upgrade FHIR versions (R4 → R4B/R5) and migrate stored resources?

A: Stored JSON must remain readable across upgrades. Experts ask for migration playbooks. Middleware facades that map from an EHR can often version the surface API without migrating a FHIR warehouse — another reason to clarify SoR vs facade early.

3.9 Why PilotFish instead of HAPI / Azure FHIR / AWS HealthLake?

Q: If I only need a FHIR repository, why not buy/run a purpose-built FHIR server?

A: Fair question. Purpose-built servers win on search depth, IG tooling, and conformance velocity. PilotFish wins when the hard problem is **integration**: many protocols (HL7v2, X12, files, vendor APIs) must become FHIR; multiple backends must look like one CapStatement; transformations and orchestration dominate; and you already standardize on PilotFish for enterprise interfaces. The sophisticated answer is often: PilotFish as the FHIR *edge and orchestration layer*, optionally backing onto a purpose-built FHIR store — or owning persistence where the resource set and search requirements are bounded and well declared.

Q: Can PilotFish sit in front of HAPI (or a cloud FHIR store) as the enterprise facade?

A: Yes — and many experts will prefer that hybrid: PilotFish for authZ enrichment, protocol mediation, non-FHIR ingress, and policy; HAPI/cloud FHIR for deep search and resource storage. This demo intentionally shows PilotFish owning the HTTP FHIR surface so you can evaluate the facade path; hybrid is a natural Phase N.

4. Decision guide — when is PilotFish viable?

Your primary need	Expert stance	PilotFish fit
Expose FHIR from many non-FHIR systems (v2, X12, vendor APIs)	Facade + mapping is the product	Strong — PilotFish's home turf
Bounded internal FHIR API (known resources, known search params)	CapStatement honesty + tests required	Viable — this demo's trajectory
Nationwide Bulk + SMART + US Core for apps / TECCA-style exchange	Deep conformance + IG program	Only with deliberate IG/SMART program or hybrid backend
General-purpose FHIR repository replacing HAPI for all apps	Experts will resist unless search/history/IG parity is proven	Weak as sole SoR today — prefer hybrid or scoped SoR
Orchestrate export, enrichment, routing across multiple FHIR stores	Integration fabric value is clear	Strong

5. Evidence this demo already gives an expert

- Working FHIR R4 CapStatement (draft) that names deferred features instead of hiding them.
- CRUD + soft delete + targeted search for core clinical resources.
- HAPI-based validation on the write path (base R4).
- Bundle transaction/batch handling.
- Keycloak OIDC Bearer integration (writes / Bulk).
- Async system \$export → status → NDJSON files (Bulk shape).
- Living automated test plan + smoke evidence under documents/.
- Route diagrams showing real PilotFish topology — not a black-box container labeled “FHIR.”

6. What to bring to the next expert review

If you are evaluating PilotFish with a FHIR architect, walk in with answers to these five:

- **SoR model:** PilotFish store, facade over EHR/CDR, or hybrid?
- **Declared surface:** exact CapStatement resource/interaction/searchParam list for go-live.
- **Conformance plan:** which Inferno/Touchstone modules, and who owns failures?
- **Security model:** SMART vs gateway-brokered OAuth; compartment enforcement design.
- **IG program:** which packages (US Core version?), terminology source, fail-open vs fail-closed.

Bottom line for experts: PilotFish is credible as a FHIR-capable integration and facade platform today, with a working expandable R4 surface demonstrated here. Whether it is viable as *your* FHIR server depends less on “can it return Patient JSON?” and more on whether you are honest about CapStatement scope, search/history needs, SMART/IG obligations, and whether persistence belongs in PilotFish or behind it. Use this demo to prove the HTTP/EIP mechanics; use the questions in §3 to size the real program.

Related demo artifacts: Capability Brief · Route Diagrams · Living Test Plan · CapStatement at `/eip/rest/fhir/metadata` · DESIGN.md honest-scope table.